OBJECTIVE

Design and implement a real-world database application using Open-Source DBMS to demonstrate database design, implementation, and application development skills.2.

ALLOWED TECH STACKDBMS:

NO SQL

Backend: Python/Django, PHP, Node.js, Java

Frontend: HTML/CSS/JS, React, Bootstrap

Hosting/Repo: GitHub + Demo Video

DELIVERABLES & MARK DISTRIBUTIONS:

S.No	Deliverable	Marks	Evaluation Points
1	**Problem Definition + SRS**	4	Domain, Scope, Users, Requirements, Feasibility
2	**ER Diagram + Schema + Normalization**	6	ERD, Tables with PK/FK, Constraints, Up to 3NF
3	**Database Implementation**	8	DDL, DML, ≥500 sample records, Views, Triggers, Procedures, Indexes
4	**SQL/NoSQL Queries + Transactions**	6	15+ Complex Queries, Joins, Aggregate, Subquery, Transaction with Commit/Rollback
5	**Application Development**	8	Web/App with DB Connectivity, CRUD, 2 Reports/Dashboards
6	**Testing + Performance Tuning**	3	Test Cases, Query Optimization, Explain Plan
7	**Report + Demo + Viva**	5	15-20 page report, 10 min PPT, Demo, Viva
	TOTAL	**40**	

EXPECTED OUTCOMES:

- After completion, students will be able to:
- Design normalized database schema for real-world problems Implement and query databases using open-source tools.
- Develop full-stack applications with DB connectivity.
- Apply transaction, security and performance concepts.
- Document and present technical work professionally

SUBMISSION GUIDELINES:

- Code: GitHub Repository Link
- DB: .sql dump file + ER Diagram
- Report: PDF, 15-20 pages with screenshots
- Demo: 3-5 min screen recording video

Open Source Database Options for PG Project:

Database | **Type** | **Setup Difficulty** | **Pros** | **Cons** | **Best Project Domains**

MongoDB | NoSQL Document | Easy | Flexible schema, JSON-like, Great with Node/Python, No joins needed | No transactions in old versions, Data redundancy | Social media, CMS, IOT, Agri-market

Neo4j | Graph DB | Medium | Best for relationships, Cypher query is easy | Different thinking than tables | Social Network, Recommendation, Fraud Detection

Redis | Key-Value | Easy | Super fast, good for caching/sessions | Data lost if no persistence, Not for main DB | Chat app, Leaderboard, Real-time dashboard

Cassandra | Columnar/NoSQL | Hard | Scales to millions of records, Distributed | Complex setup, No joins | Big Data, Logging, Sensor/IOT data

---

Mandatory for All Choices:

- Must be Open Source and Free

- Must provide: DB dump, ERD/Schema diagram, Screenshots

- Must connect to a Frontend application. DB-only projects = max 20 marks.

Review-1 deliverables:

1. 1_ProblemStatement.pdf, 2_SRS.pdf,
2. 3_ERD.png, 4_Schema_Design.pdf - Tables with PK,FK, Constraints, up to 3NF
3. 5_TechStack.pdf - DB, Language, Tools.

Review-2 deliverables:

1. DB_Dump.sql, Screenshots of ≥500 records, Views, Triggers, Procedures, Indexes.
2. 6_Queries.pdf - 15+ Complex Queries with output, 7_TestCases.xlsx.
3. GitHub Link + 2 min screen recording of CRUD working. Backend + DB Connectivity done.

Review-3 deliverables:

1. Final app / web pages with all CRUD, Min 5 pages. 3-5 min DemoVideo.mp4.
2. 8_FinalReport.pdf 15-20 pages, 9_PPT.pptx, Live Demo + Viva.

Formatting Guidelines for Students

1. *Font*: Times New Roman 12, 1.5 line spacing
2. *Margins*: 1 inch all sides
3. *Screenshots*: Must have caption and figure number
4. *Page Count*: 15-20 pages excluding Certificate, Declaration, References
5. *Plagiarism*: <15%

VELLORE INSTITUTE OF TECHNOLOGY

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

PG LEVEL PROJECT REPORT

ON

<PROJECT TITLE IN CAPS>

Submitted by

<STUDENT NAME> - <REG NO>

<STUDENT NAME> - <REG NO>

Under the guidance of

<FACULTY NAME>

<DESIGNATION>

Month Year

TABLE OF CONTENTS

CERTIFICATE i

DECLARATION ii

ACKNOWLEDGEMENT iii

ABSTRACT iv

LIST OF FIGURES v

LIST OF TABLES vi

1 INTRODUCTION 1

1.1 Project Overview 1

1.2 Problem Statement 1

1.3 Objectives 2

1.4 Scope of the Project 2

1.5 Organization of Report 2

2 LITERATURE SURVEY 3

2.1 Existing Systems 3

2.2 Limitations of Existing Systems 4

2.3 Proposed System 4

3 SYSTEM REQUIREMENTS 5

3.1 Functional Requirements 5

3.2 Non-Functional Requirements 5

3.3 Hardware & Software Requirements 6

4 SYSTEM DESIGN 7

4.1 ER Diagram 7

4.2 Database Schema Design 8

4.3 Normalization - up to 3NF 9

4.4 Data Dictionary 9

5 IMPLEMENTATION 10

5.1 Database Implementation - DDL, DML 10

5.2 Views, Triggers, Stored Procedures 11

5.3 Complex SQL/NoSQL Queries 12

5.4 Transaction Management 12

5.5 Application Development - Frontend + Backend 13

5.6 Screenshots of Modules 14

6 TESTING & PERFORMANCE TUNING 15

6.1 Test Cases 15

7 RESULTS AND DISCUSSION 17

7.1 Sample Reports / Dashboards 17

8 CONCLUSION AND FUTURE WORK 18

REFERENCES 19

APPENDICES 20

A. Source Code Snippets

B. DB Dump

C. GitHub Link